

ECE 276B Project 3: Infinite-Horizon Stochastic Optimal Control

Winston Chou

University of California, San Diego

PID: A17460970

La Jolla, CA, USA

w3chou@ucsd.edu

Abstract—This paper presents implementations of search-based motion planning algorithms for solving infinite-horizon stochastic optimal control problems in 3D euclidean spaces. Two major algorithms are implemented: A* (A-star) and LPA* (Lifelong Planning A*). The results show that different space resolutions, heuristic weights, and planning strategies can significantly impact the performance of these algorithms in complex environments. The approach can provide optimal shortest paths for robotic navigation and planning problems.

Index Terms—infinite-horizon stochastic optimal control, A*, Lifelong Planning A*, motion planning, replanning, robotics

I. INTRODUCTION

Motion planning is a fundamental challenge in robotics, particularly for autonomous agents operating in complex three-dimensional environments. The primary goal is to find a collision-free path from an initial configuration to a target location while optimizing certain criteria, such as path length or time. In 3D spaces, the increased dimensionality and presence of diverse obstacles like narrow passages and varying heights necessitate robust and efficient planning algorithms.

A. Problem Overview

This project focuses on implementing and evaluating search-based motion planning algorithms in various 3D grid-map environments. We consider a set of diverse scenarios, including a "Flappy Bird" style obstacle course, a dense 3D maze, and environments with dynamic target sequences such as "Window" and "Room". The agent must navigate these spaces by discretizing the continuous 3D volume into a grid and performing graph searches to identify optimal trajectories.

B. Approach

To address these challenges, we employ two powerful search-based algorithms: A* and **Lifelong Planning A* (LPA*)**.

The A* algorithm is used for static environments where the entire map and goal are known beforehand. It combines the cost-to-come with a heuristic estimate of the cost-to-go, ensuring optimality while maintaining efficiency. For environments with dynamic targets or changing goal locations—where re-calculating the entire path from scratch would be computationally expensive—we utilize LPA*. LPA* is an incremental search algorithm that reuses information from

previous searches to update the current path efficiently as the target location or environment changes.

By benchmarking these approaches across different grid resolutions and heuristic weights (ϵ), we analyze the trade-offs between path quality and computational overhead.

II. PROBLEM STATEMENT

The goal of 3D motion planning is to find an optimal collision-free path for a point robot operating in a 3D Euclidean space. The problem can be defined by the environment boundaries, a set of obstacles, a starting configuration, and one or more goal configurations.

A. State Space and Environment

The configuration space $\mathcal{X}_{free} \subset \mathbb{R}^3$ is defined within a bounding box $\mathcal{B} = [x_{min}, x_{max}] \times [y_{min}, y_{max}] \times [z_{min}, z_{max}]$. The environment contains a set of static obstacles $\mathcal{O} = \{O_1, O_2, \dots, O_n\}$, where each O_i is represented as an Axis-Aligned Bounding Box (AABB). The free space is thus $\mathcal{X}_{free} = \mathcal{B} \setminus \bigcup_{i=1}^n O_i$.

To solve the planning problem using search-based methods, the continuous space is discretized into a uniform grid with a specified resolution res . Each state $x \in \mathcal{S}$ represents a coordinate $(x_{pos}, y_{pos}, z_{pos})$ in the grid.

B. Action Space and Connectivity

The robot can move between adjacent grid cells. We employ a 26-connectivity model, where each node is connected to all its neighbors in a $3 \times 3 \times 3$ cube centered at the node (excluding the node itself). This allows for diagonal movements in 3D, providing more flexible and shorter paths than a simple 6-connectivity (cardinal directions) model.

The cost of moving from state x to an adjacent state x' is defined by the Euclidean distance:

$$\begin{aligned} c(x, x') &= \|x - x'\|_2 \\ &= \sqrt{(x_{pos} - x'_{pos})^2 + (y_{pos} - y'_{pos})^2 + (z_{pos} - z'_{pos})^2} \end{aligned} \quad (1)$$

C. Collision Constraints

A path ρ is valid if it is entirely contained within $\mathcal{X}_{free}$. Since planning is performed on a discrete graph, we must ensure that the linear segments connecting discrete states do

not intersect any obstacle O_i . This is handled by a safety evaluator that performs intersection tests between the path segments and the AABBs in the environment.

D. Objective Function

The objective is to find a path $\rho = (x_0, x_1, \dots, x_k)$ from a start state s to a goal state τ that minimizes the cumulative cost:

$$J(\rho) = \sum_{i=0}^{k-1} c(x_i, x_{i+1}) \quad (2)$$

subject to $x_0 = s$, $x_k = \tau$, and $x_i \in \mathcal{S}_{free}$ for all i . For dynamic target scenarios, the agent must visit a sequence of goals $T = \{\tau_1, \tau_2, \dots, \tau_m\}$ in order, minimizing the cost for each segment while potentially adapting to newly revealed goal locations.

E. Static Benchmarking Environments

To evaluate the performance of our search-based planners, we utilize five distinct static maps:

- **Flappy Bird (E1):** Features vertical pillars with narrow openings at varying heights, requiring precise elevation control.
- **Maze (E2):** A dense 3D maze with numerous narrow passages and dead ends, testing search efficiency.
- **Monza (E3):** A winding, corridor-like environment inspired by a race track, challenging the planner's ability to navigate smooth turns.
- **Single Cube (E4):** A basic configuration with a single obstacle, used for initial verification.
- **Tower (E5):** Stacked structures creating high vertical obstacles that necessitate maneuvers in the Z -axis.

F. Dynamic Target Stay Time Requirements

In scenarios with dynamic targets, each goal τ_i in the sequence T remains active for a specific *stay time* $t_{stay,i}$ before moving to the next position τ_{i+1} . A plan for the sequence is considered successful only if the motion planner can compute a valid path to each goal before the respective stay time expires. Specifically, the cumulative planning time must satisfy:

$$\sum_{j=1}^i t_{plan,j} \leq \sum_{j=1}^i t_{stay,j} \quad (3)$$

Furthermore, the system follows a synchronous update rule: if the planning for goal τ_i completes within a time $t_{plan,i} < t_{stay,i}$, the system waits until the full duration $\sum_{j=1}^i t_{stay,j}$ has elapsed before initiating the search for the next goal τ_{i+1} . This ensures that the agent's path updates are synchronized with the discrete jumps of the dynamic target.

For the benchmarking environments used in this study, the stay times are configured as follows:

- **Window (E6):** The target stays at the first location for 3 seconds ($t_{stay,1} = 3s$) and subsequent locations for 1 second each ($t_{stay,i>1} = 1s$).

- **Room (E7):** The target stays for 0.3 seconds initially ($t_{stay,1} = 0.3s$) and 0.1 seconds for all following goals ($t_{stay,i>1} = 0.1s$).

These tight timing constraints emphasize the need for efficient replanning algorithms like LPA* that can reuse previous computations to minimize t_{plan} .

III. TECHNICAL APPROACH

The motion planning problem is solved using search-based algorithms on a discretized 3D grid. The implementation focuses on safety, efficiency, and adaptability to dynamic targets.

A. Continuous Collision Detection with Coal

Safety is ensured by validating that each linear segment between discrete grid states does not intersect any obstacles. We utilize the **coal** library (a fork of the Flexible Collision Library (FCL) formerly known as *hpp-fcl*), which provides high-performance proximity queries.

Each static obstacle O_i is represented as a `coal::CollisionObject` with a `Box` geometry. For a linear traversal between states x_i and x_{i+1} , we model the path segment as a `coal::Capsule` with a radius of zero. This representation allows the library to perform continuous collision detection (CCD) between the segment and the Axis-Aligned Bounding Boxes (AABBs) of the obstacles. This approach ensures that even diagonal movements are strictly collision-free.

B. Optimized A* Algorithm

For static environments, we implement the A* algorithm to find the optimal path ρ^* . A* maintains an OPEN list (priority queue) of discovered nodes, ordered by the evaluation function $f(x) = g(x) + \epsilon h(x)$, where $g(x)$ is the cost-to-come and $h(x)$ is the heuristic estimate (Euclidean distance) to the goal τ .

To maximize computational efficiency, we pre-calculate the neighbor offsets and their corresponding Euclidean costs for the 26-connectivity model:

$$\Delta_{neighbors} = \{(\delta_x, \delta_y, \delta_z) \mid \delta_i \in \{-1, 0, 1\}, \sum |\delta_i| > 0\} \quad (4)$$

By caching these relative indices and distances as `INDEX_OFFSETS`, the planner avoids redundant arithmetic during the `get_neighbors` call, significantly reducing the overhead of graph expansion.

C. Dynamic Replanning with Lifelong Planning A*

In scenarios where the target τ moves through a sequence $\{\tau_1, \tau_2, \dots, \tau_m\}$, re-running A* from scratch for every segment is computationally prohibitive. We employ **Lifelong Planning A* (LPA*)** [2] to exploit the fact that large portions of the search graph remain valid between subsequent planning iterations.

LPA* maintains two cost estimates for each node: the g -value (cost-to-come) and the v -value (cost-to-come during the last expansion). A node is *inconsistent* if $g(x) \neq v(x)$. The

Algorithm 1 A* Search Algorithm

```
1: Initialize:
2:  $g(s) \leftarrow 0, h(s) \leftarrow \text{heuristic}(s, \tau, \epsilon), f(s) \leftarrow g(s) + h(s)$ 
3:  $g(x) \leftarrow \infty$  for all  $x \in \mathcal{X} \setminus \{s\}$ 
4:  $OPEN \leftarrow \{s\}$  with priority  $f(s)$ 
5: while  $OPEN \neq \emptyset$  do
6:    $x \leftarrow$  pop node with min  $f(x)$  from  $OPEN$ 
7:   mark  $x$  as closed
8:   if  $\text{grid}(x) = \text{grid}(\tau)$  then
9:     return backtrack path from  $x$ 
10:  end if
11:  for each neighbor  $x'$  of  $x$  do
12:     $cost \leftarrow c(x, x')$ 
13:    if  $x'$  not opened or  $g(x) + cost < g(x')$  then
14:       $parent(x') \leftarrow x$ 
15:       $g(x') \leftarrow g(x) + cost$ 
16:      if  $h(x')$  is not computed then
17:         $h(x') \leftarrow \text{heuristic}(x', \tau, \epsilon)$ 
18:      end if
19:       $f(x') \leftarrow g(x') + h(x')$ 
20:      mark  $x'$  as opened
21:      insert/update  $x'$  in  $OPEN$  with priority  $f(x')$ 
22:    end if
23:  end for
24: end while
25: return failure
```

algorithm iteratively expands inconsistent nodes to maintain the shortest path tree.

A key modification in our implementation addresses the frequent movement of the goal τ . When the goal shifts from τ_i to τ_{i+1} , the heuristic values $h(x, \tau)$ —and consequently the priority keys $k(x)$ —for all nodes in the $OPEN$ list become outdated. Unlike standard LPA*, our system explicitly updates the heuristics and keys for every node remaining in the $OPEN$ list before initiating the search for a new goal. This ensures that the search remains focused on the new target while reusing the previously explored connectivity information.

IV. RESULTS

The proposed algorithms were evaluated across seven diverse 3D environments. The benchmarks show that A* provides reliable shortest paths in static maps like "Flappy Bird" and "Maze," while LPA* demonstrates superior performance in dynamic scenarios by reducing the number of node expansions required to reach new goal locations.

A. Static Map Benchmarks

In static environments, the choice of resolution and heuristic weight ϵ (for Weighted A*) creates a trade-off between path optimality and planning speed. Table I shows the impact of ϵ on path length and expansion count, while Table II highlights the sensitivity of the planner to grid resolution. The visual results of the planned paths for environments E1–E5 are shown in Fig. ??, ??, ??, ??, and ?? respectively.

Algorithm 2 Lifelong Planning A* (LPA*)

```
1: procedure CalculateKey( $x$ )
2: if  $h(x)$  is not computed then
3:    $h(x) \leftarrow \text{heuristic}(x, \tau, \epsilon)$ 
4: end if
5: return  $[\min(g(x), v(x)) + h(x); \min(g(x), v(x))]$ 
6:
7: procedure UpdateMembership( $x$ )
8: if  $v(x) \neq g(x)$  and  $x$  is not closed then
9:   mark  $x$  as opened,  $OPEN[x] \leftarrow \text{CalculateKey}(x)$ 
10: else if  $x$  is opened then
11:   remove  $x$  from  $OPEN$ 
12: end if
13:
14: procedure ComputePath()
15: while  $TopKey(OPEN) < \text{CalculateKey}(\tau) \vee v(\tau) \neq g(\tau)$  do
16:    $x \leftarrow$  pop node with min key from  $OPEN$ 
17:   if  $v(x) > g(x)$  then
18:      $v(x) \leftarrow g(x)$ , mark  $x$  as closed and not opened
19:     for all neighbor  $x'$  of  $x$  do
20:       if  $g(x') > g(x) + c(x, x')$  then
21:          $g(x') \leftarrow g(x) + c(x, x')$ ,  $parent(x') \leftarrow x$ 
22:         UpdateMembership( $x'$ )
23:       end if
24:     end for
25:   else
26:      $v(x) \leftarrow \infty$ , UpdateMembership( $x$ )
27:     for all neighbor  $x'$  of  $x$  s.t.  $parent(x') = x$  do
28:        $g(x') \leftarrow \min_{x'' \in Pred(x')}(v(x'') + c(x'', x'))$ 
29:        $parent(x') \leftarrow \text{argmin}_{x'' \in Pred(x')}(v(x'') + c(x'', x'))$ 
30:       UpdateMembership( $x'$ )
31:     end for
32:   end if
33: end while
34:
35: Main Loop:
36:  $v(x), g(x) \leftarrow \infty$  for all  $x$ ;  $g(s) \leftarrow 0$ ;
   UpdateMembership( $s$ )
37: loop
38:   if target  $\tau$  moved then
39:     reset all  $h(x) \leftarrow \infty$ 
40:     for all  $x \in OPEN$  do
41:       update  $OPEN[x] \leftarrow \text{CalculateKey}(x)$ 
42:     end for
43:   end if
44:   ComputePath()
45:   return backtrack path from  $\tau$ 
46:   wait for next goal  $\tau$  or obstacle update
47: end loop
```

1) *Impact of Resolution:* Decreasing the grid resolution (e.g., from 1.0 to 0.25) significantly improves path optimality but at a high computational cost. As observed in Fig. ??, a coarser resolution of 1.0 causes the A* algorithm to fail in the dense Maze (E2) because narrow passages become impassable in the discretized graph. In contrast, the 0.25 resolution path is smoother and shorter but requires nearly 100× more node expansions (Table II).

2) *Impact of Heuristic Weight (ϵ):* The use of Weighted A* ($\epsilon > 1$) acts as a greedy search bias. In environments like Flappy Bird (E1) and Tower (E5), increasing ϵ from 1.0 to 2.0 reduces node expansions by up to 60% (Table I) with only a marginal increase in path length (< 5%). Visually, the $\epsilon = 2.0$ paths in Fig. ?? and ?? exhibit more jagged segments as the planner prioritizes reaching the goal over finding the absolute shortest traversal.

TABLE I: A* Heuristic Weights (Res=0.5)

Env	Planner	Res	ϵ	Success	Length	Time (s)	Nodes
1	A*	0.5	1.0	Yes	25.59	1.4909	5944
1	A*	0.5	2.0	Yes	26.52	0.3730	2209
2	A*	0.5	1.0	Yes	79.29	3.6025	19493
2	A*	0.5	2.0	Yes	80.95	2.6808	15822
3	A*	0.5	1.0	Yes	78.03	1.3350	5984
3	A*	0.5	2.0	Yes	78.19	1.0777	5102
4	A*	0.5	1.0	Yes	8.32	0.0719	452
4	A*	0.5	2.0	Yes	8.32	0.0104	190
5	A*	0.5	1.0	Yes	32.80	1.1584	5582
5	A*	0.5	2.0	Yes	33.97	0.2712	2052

TABLE II: A* Resolution Comparison ($\epsilon = 1.0$)

Env	Planner	Res	ϵ	Success	Length	Time (s)	Nodes
1	A*	0.25	1.0	Yes	25.59	11.0231	33685
1	A*	0.5	1.0	Yes	25.59	1.4909	5944
1	A*	1.0	1.0	Yes	25.96	0.3014	1373
2	A*	0.25	1.0	Yes	76.23	41.3566	122067
2	A*	0.5	1.0	Yes	79.29	3.6025	19493
2	A*	1.0	1.0	No	0.00	0.0120	180
3	A*	0.25	1.0	Yes	76.61	12.0423	36176
3	A*	0.5	1.0	Yes	78.03	1.3350	5984
3	A*	1.0	1.0	Yes	81.41	0.1520	1261
4	A*	0.25	1.0	Yes	8.15	0.3853	1257
4	A*	0.5	1.0	Yes	8.32	0.0719	452
4	A*	1.0	1.0	Yes	8.59	0.0620	354
5	A*	0.25	1.0	Yes	29.64	10.6647	30328
5	A*	0.5	1.0	Yes	32.80	1.1584	5582
5	A*	1.0	1.0	Yes	35.87	0.1718	1278

B. Dynamic Map Benchmarks

For environments like "Room" and "Window" with dynamic targets, LPA* effectively manages the tight stay-time constraints. As shown in Table III, by reusing the search graph, the planner finds paths to subsequent goals significantly faster than A* while maintaining path quality. Figures ?? and ?? visualize the resulting trajectories in the Window and Room environments.

1) *A* vs. LPA* Performance:* The primary advantage of LPA* is its ability to minimize replanning latency by reusing previous g -value computations. In the Window environment (E6), while A* must re-expand thousands of nodes for every

target move, LPA* often performs zero expansions (Table III) when the target stays within a previously explored region. This is reflected in the cumulative trajectories in Fig. ??, where LPA* maintains a coherent search tree across the window opening.

2) *Meeting Tight Timing Constraints:* In the Room environment (E7), the target stay times are extremely short (0.1s). Static A* struggles to find paths within this window at high resolutions. However, LPA* maintains sub-millisecond planning times after the initial search. As illustrated in Fig. ??, LPA* successfully tracks all 8 goal locations in the room, whereas A* results in frequent "Success: No" markers in the logs when the stay time expires before convergence.

TABLE III: Dynamic Environment: A* vs LPA*

Env	Planner	Res	ϵ	Success	Length	Time (s)	Nodes
6	LPA*	0.25	1.0	No	166.37	15.2089	40532
6	A*	0.5	1.0	No	168.66	9.4852	34371
6	LPA*	0.5	1.0	Yes	168.11	1.9411	7253
6	A*	0.5	2.0	Yes	173.60	0.5006	5006
6	LPA*	0.5	2.0	Yes	195.95	0.0822	851
6	A*	1.0	1.0	Yes	171.39	1.2661	7348
6	LPA*	1.0	1.0	Yes	170.59	0.2612	1429
7	LPA*	0.25	1.0	No	66.35	2.5726	7824
7	A*	0.5	1.0	No	69.38	0.9725	5350
7	LPA*	0.5	1.0	Yes	69.38	0.3713	1549
7	A*	0.5	2.0	Yes	70.14	0.3811	3008
7	LPA*	0.5	2.0	Yes	73.42	0.1382	1035
7	A*	1.0	1.0	No	66.01	0.3869	3384
7	LPA*	1.0	1.0	Yes	132.52	0.0476	509

V. CONCLUSION

In this work, we implemented and evaluated search-based motion planning algorithms in 3D environments. By integrating the *coal* library for continuous collision detection and optimizing graph expansion through neighbor caching, we achieved efficient path computation in complex static maps. Furthermore, the adaptation of LPA* to handle dynamic goal positions by updating heuristic values and priority keys proved effective in meeting tight stay-time constraints across diverse benchmarking scenarios. Future work could explore more advanced heuristics or sampling-based methods to further improve performance in extremely dense or unstructured 3D environments.

REFERENCES

- [1] N. Atanasov, "ECE276B: Planning & Learning in Robotics – Schedule," University of California San Diego, [Online]. Available: <https://natanaso.github.io/ece276b/schedule.html>. [Accessed: May 22, 2026].
- [2] S. Koenig, M. Likhachev and D. Furcy, "Lifelong Planning A*," Artificial Intelligence, vol. 155, no. 1-2, pp. 93-146, 2004.